Objective-1: Check use case of table partitioning using MySQL.

Motivation:
Consider the following citizen table:

```
CREATE TABLE citizen (
    nid INT NOT NULL,
    name TEXT NOT NULL,
    dob DATE NOT NULL,
    religion TEXT NOT NULL,
    pob TEXT NOT NULL
);
```

Storing hundreds of citizens into this table might be an issue in the first place. But for a country like Bangladesh where we have more than 20 crore population, this might become a nightmare if we store all records in a single table.

Even with proper indexing searching for a particular citizen might take a few seconds or minutes at times which might not be expected at all in any scenario. And we are not talking about a single query a day, rather there might be millions of active queries each minute.

Solution:
This is when table partitioning comes handy. In MySQL we can partition a table in below ways:

- LIST
- RANGE
- KEY
- HASH
- COLUMNS

In this lab we will try out some partitioning types and see how data is distributed across the partitions. To know more about table partitioning MySQL Official Documentation on Partitioning is the best place to start.

Case#01: Data Distribution Using List Pattern:

One of our DBAs started thinking like Radcliffe. He planned to distribute the citizen data based on their religion. For him this will be a simple partition of 5 with following categories:

Religion
Islam
Hinduism

Buddhism
Christianity
Other

```
CREATE TABLE citizen (
nid INT NOT NULL,
name VARCHAR(100) NOT NULL,
dob DATE NOT NULL,
religion VARCHAR(50) NOT NULL,
pob VARCHAR(100) NOT NULL
)
PARTITION BY LIST COLUMNS(religion) (
PARTITION islam VALUES IN ('Islam'),
PARTITION hinduism VALUES IN ('Hinduism'),
PARTITION buddhism VALUES IN ('Buddhism'),
PARTITION christianity VALUES IN ('Christianity'),
PARTITION others VALUES IN ('Other')
);
```

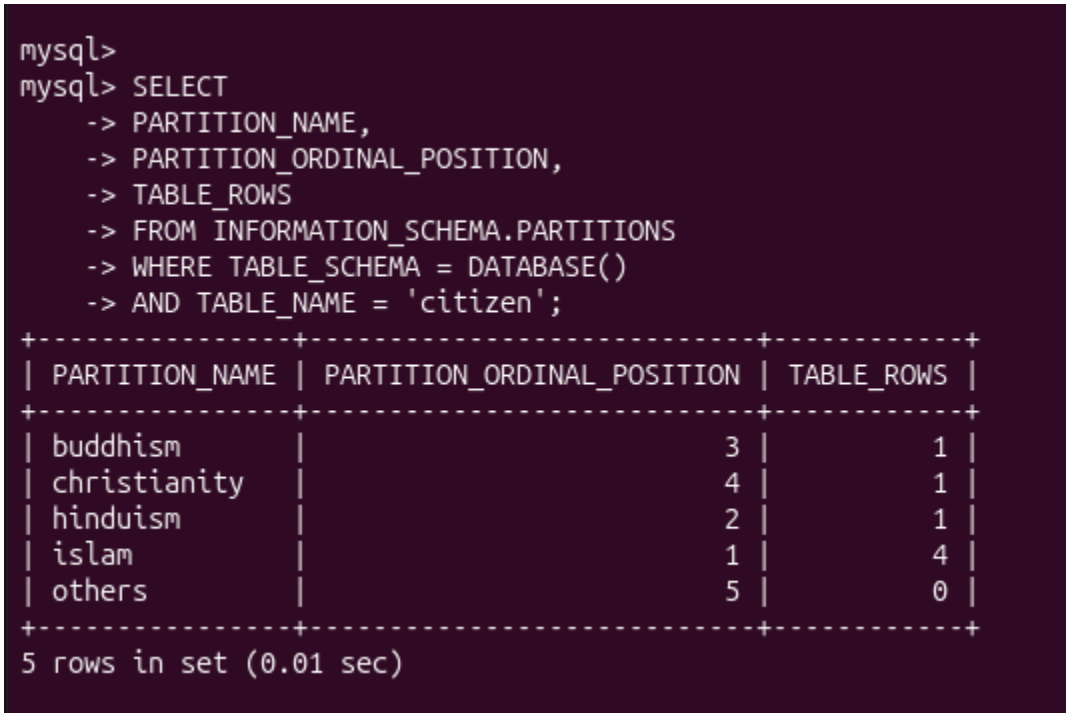
Data Insertion:

```
INSERT INTO citizen VALUES
(1,'Abdul Karim','1938-05-12','Islam','Dhaka'),
(2,'Rahim Uddin','1955-03-10','Islam','Dhaka'),
(3,'Rudra Sarkar','1978-08-15','Hinduism','Khulna'),
(4,'Shormi Barua','1992-05-20','Buddhism','Dhaka'),
(5,'Nabila Sultana','2005-11-01','Islam','Rajshahi'),
(6,'Hasan Mahmud','2018-07-12','Islam','Dhaka'),
(7,'Rahul Rema','2026-01-15','Christianity','Chattogram');
```

See Where Your Data Goes:

Query	You want to validate data distribution

SQL	`SELECT`
	`PARTITION_NAME,`
	`PARTITION_ORDINAL_POSITION,`
	`TABLE_ROWS`
	`FROM INFORMATION_SCHEMA.PARTITIONS`

	`WHERE TABLE_SCHEMA = DATABASE()` `AND TABLE_NAME = 'citizen';`
Result	

```
mysql>
mysql> SELECT
    -> PARTITION_NAME,
    -> PARTITION_ORDINAL_POSITION,
    -> TABLE_ROWS
    -> FROM INFORMATION_SCHEMA.PARTITIONS
    -> WHERE TABLE_SCHEMA = DATABASE()
    -> AND TABLE_NAME = 'citizen';
+----------------+----------------------------+------------+
| PARTITION_NAME | PARTITION_ORDINAL_POSITION | TABLE_ROWS |
+----------------+----------------------------+------------+
| buddhism       |                          3 |          1 |
| christianity   |                          4 |          1 |
| hinduism       |                          2 |          1 |
| islam          |                          1 |          4 |
| others         |                          5 |          0 |
+----------------+----------------------------+------------+
5 rows in set (0.01 sec)
```

Case#02: Data Distribution Using Range Pattern:

If you look closely, although list partitioning may have its other use cases, in our case **the data is not uniformly distributed.** You know, **dividing things based on religion never helps in this sub continent.** So in this case we can take help of another approach of partitioning, and yes, you guessed it right, using range. We will divide the data distribution based on the generation category. Usually we know generations can be categorized like below table (Wiki: Generation Category):

Birth Year	Generation
<1946	Silent Generation
1946-1964	Baby Boomers
1965-1980	Generation X
1981-1996	Millennials
1997-2021	Generation Z
2013-2024	Generation Alpha

2025+	Generation Beta

```
CREATE TABLE citizen (
    nid INT NOT NULL,
    name TEXT NOT NULL,
    dob DATE NOT NULL,
    religion TEXT NOT NULL,
    pob TEXT NOT NULL
)
PARTITION BY RANGE (YEAR(dob)) (
    PARTITION silent_gen VALUES LESS THAN (1946),
    PARTITION baby_boomers VALUES LESS THAN (1965),
    PARTITION gen_x VALUES LESS THAN (1981),
    PARTITION millennials VALUES LESS THAN (1997),
    PARTITION gen_z VALUES LESS THAN (2013),
    PARTITION gen_alpha VALUES LESS THAN (2025),
    PARTITION gen_beta VALUES LESS THAN MAXVALUE
);
```

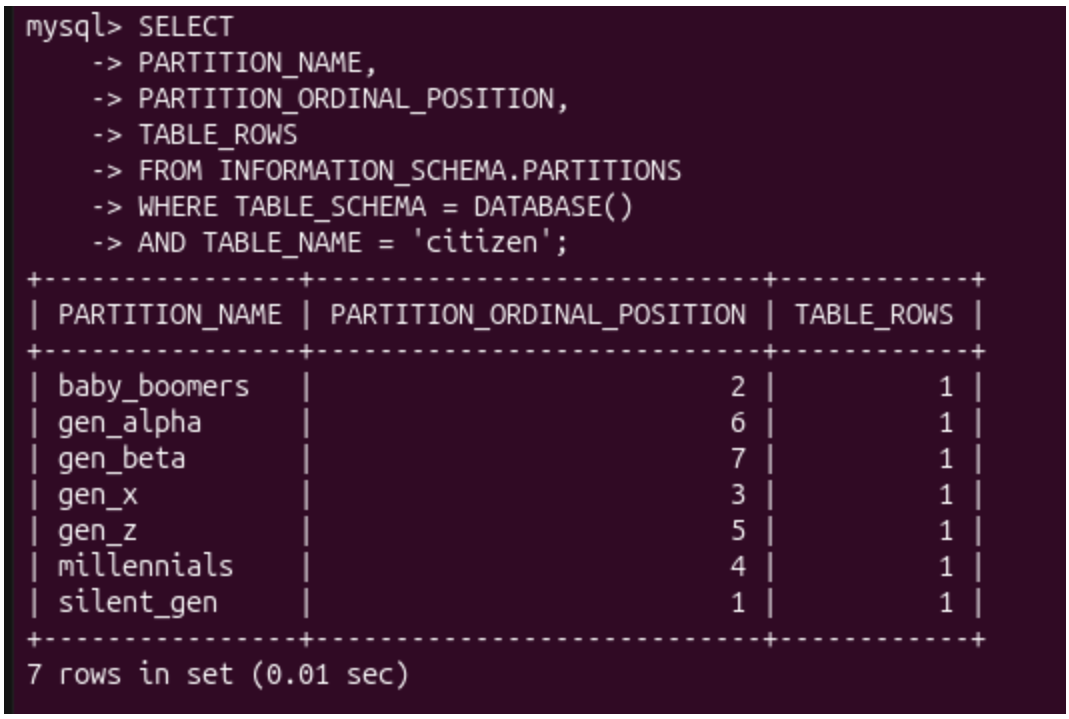
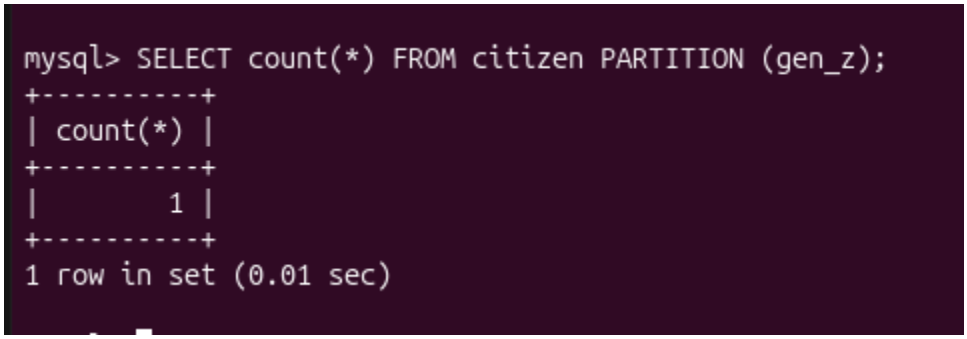
Data Insertion:

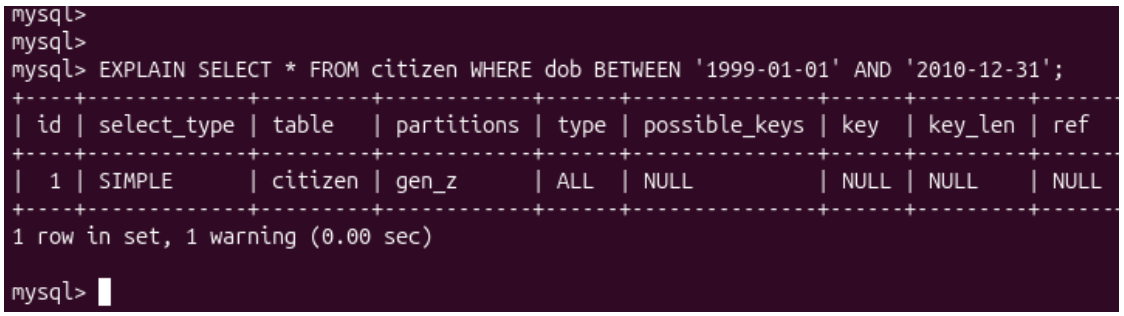
```
INSERT INTO citizen VALUES
(1,'Abdul Karim','1938-05-12','Islam','Dhaka'),      -- Silent Generation
(2,'Rahim Uddin','1955-03-10','Islam','Dhaka'),      -- Baby Boomers
(3,'Rudra Sarkar','1978-08-15','Hindu','Khulna'),     -- Generation X
(4,'Ayesha Akter','1992-05-20','Islam','Dhaka'),     -- Millennials
(5,'Nabila Sultana','2005-11-01','Islam','Rajshahi'),-- Generation Z
(6,'Hasan Mahmud','2018-07-12','Islam','Dhaka'),     -- Generation Alpha
(7,'Rahul Rema','2026-01-15','Chirstian','Chattogram');-- Generation Beta
```

See Where Your Data Goes:

Query	You want to validate data distribution

SQL	`SELECT`
	`PARTITION_NAME,`
	`PARTITION_ORDINAL_POSITION,`
	`TABLE_ROWS`
	`FROM INFORMATION_SCHEMA.PARTITIONS`
	`WHERE TABLE_SCHEMA = DATABASE()`
	`AND TABLE_NAME = 'citizen';`

Result	
Query	You want to know how many gen-z you have deal with currently in Bangladesh
SQL	`SELECT count(*) FROM citizen PARTITION (gen_z);`
Result	
Query	You want to know how which generation (partition) a particular DOB (data) falls in
SQL	`EXPLAIN SELECT * FROM citizen WHERE dob BETWEEN '1999-01-01' AND '2010-12-31';`

Result (Truncated Snapshot)	

```
mysql>
mysql>
mysql> EXPLAIN SELECT * FROM citizen WHERE dob BETWEEN '1999-01-01' AND '2010-12-31';
+----+-------------+---------+------------+------+---------------+------+---------+-------
| id | select_type | table   | partitions | type | possible_keys | key  | key_len | ref
+----+-------------+---------+------------+------+---------------+------+---------+-------
|  1 | SIMPLE      | citizen | gen_z      | ALL  | NULL          | NULL | NULL    | NULL
+----+-------------+---------+------------+------+---------------+------+---------+-------
1 row in set, 1 warning (0.00 sec)

mysql>
```

Your Turn:

We have a gut feeling that even though the data records seem to be distributed uniformly for now, this approach might not help us for long, because within the next 10 years or so more and more data records will be populated in the **gen_beta** partition. On top of that max of our data queries will happen on **millennials, gen_z, gen_alpha and gen_beta partitions** as they are the most active citizens right now. So what might be a good partition approach if this happened to be 2000-2010, is not the same for now and future. So, we have decided to distribute the data using place of birth (POB). Create a table named citizen with the same attributes as before but using **pob** as the main consideration for partitioning.

Objective-2: Check ACID compliances in DBMS using Transaction in MySQL.

Environment Creation:

```
CREATE TABLE bank_account (
    account_id INT PRIMARY KEY,
    customer_name VARCHAR(50),
    balance DECIMAL(10,2)
);
```

Data Insertion:

```
INSERT INTO bank_account VALUES
(1, 'Md. Golam Hossain', 10000),
(2, 'Md. Golam Rabbani', 8000);
```

Hands-on:

- open command line	
- go to mysql bin directory	`cd C:\xampp\mysql\bin`
- run mysql client	`mysql -u root`
- do the same for another transaction section	

Run-queries:

TXN-1	**TXN-2**
Atomicity	

`BEGIN;`
 	
`UPDATE bank_account`
`SET balance = balance - 5000`
`WHERE account_id = 1;`
 	
`UPDATE bank_account`
`SET balance = balance + 5000`
`WHERE account_id = 2;`
	`BEGIN;`
	
	`UPDATE bank_account`
	`SET balance = balance - 5000`
	`WHERE account_id = 1;`
	
	`ROLLBACK;`

COMMIT;	
Consistency	
SELECT SUM(balance) AS total_balance FROM bank_account;	SELECT SUM(balance) AS total_balance FROM bank_account;
Isolation	
START TRANSACTION; SELECT * FROM lab9.bank_account; UPDATE lab9.bank_account SET balance = balance - 1000 WHERE account_id = 2; SELECT * FROM lab9.bank_account; UPDATE lab9.bank_account SET balance = balance + 1000 WHERE account_id = 1; SELECT * FROM lab9.bank_account; COMMIT;	START TRANSACTION; SELECT * FROM lab9.bank_account; UPDATE lab9.bank_account SET balance = balance - 2000 WHERE account_id = 1; SELECT * FROM lab9.bank_account; UPDATE lab9.bank_account SET balance = balance + 2000 WHERE account_id = 2; SELECT * FROM lab9.bank_account; COMMIT;
Durability	
START TRANSACTION; SELECT * FROM lab9.bank_account; UPDATE lab9.bank_account SET balance = balance - 1000 WHERE account_id = 2; SELECT * FROM lab9.bank_account; ROLLBACK;	Restart Xampp at this point and see.